

Proyecto Base de Datos I — Hito 2

Implementación, Optimización y Experimentación con Índices

Sergio Delgado Antonio Guerrero Luciana Melgarejo

Plataforma de catálogo literario

Ciclo 2026.1

Agenda

- 1 El proyecto
- 2 Implementación
- 3 Experimentación
- 4 Conclusiones

Plataforma de catálogo literario:

- Entidad central: Material (libros, ensayos, revistas, poemas, audiolibros).
- Relacionada con autores, géneros, subgéneros, editoriales, premios.
- Interacciones de usuario: Likes, Leer (lecturas), Resena.

Del Hito 1 al Hito 2:

- Hito 1: modelo E-R y relacional.
- Hito 2: implementación física + experimento de índices.

25 tablas en total: maestras, débiles, subtipos y tablas de relación.

Tres mecanismos para las restricciones del Hito 1:

- **Constraints** de columna/tabla: PK, FK, UNIQUE, NOT NULL, CHECK.
- **Discriminador** Tipo en Material: herencia total y disjunta.
- **Triggers** para reglas multi-tabla (4 funciones):
 - Año de publicación $\geq$ fundación de la editorial.
 - Fecha de reseña $\geq$ año del material.
 - Todo material con ≥ 1 autor (*trigger diferido*).
 - Todo género con ≥ 1 autor representante (*trigger diferido*).

Más **5 vistas** de usuario que encapsulan los JOIN frecuentes.

Cambios respecto al Hito 1

Cerramos reglas que el modelo relacional dejaba sin enforcement, sin reinterpretar el dominio:

- **[A]** Discriminador Tipo; **[B][C]** triggers de participación total.
- **[D]** Fecha en PK de Leer (permite relecturas).
- **[I]** Nueva tabla PerteneceSubGenero (consulta "por subgénero" que el Hito 1 pedía pero el modelo dejaba imposible).
- **[J]** Reescritura de Q3 para evitar un producto cartesiano.
- **[K]** Quitamos índices redundantes con la PK.

(11 cambios documentados en docs/cambios_respecto_hito1.md)

Datos: 4 escenarios reproducibles

Generador propio (Python + Faker), carga por COPY, semilla fija.

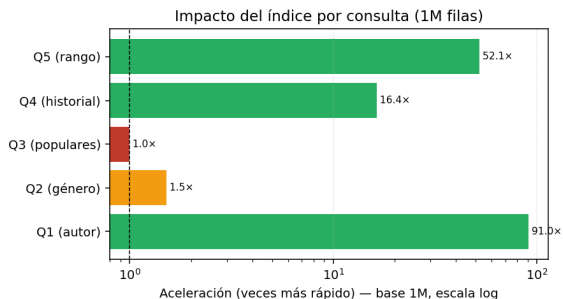
Base	Materiales	Likes/Lecturas/Reseñas	Total filas
1k	300	1 000 c/u	~5 mil
10k	2 000	10 000 c/u	~46 mil
100k	20 000	100 000 c/u	~484 mil
1m	100 000	1 000 000 c/u	~3.86 millones

Las 5 consultas y la metodología

ID	Qué responde	Patrón
Q1	Materiales de un autor	igualdad selectiva
Q2	Materiales de un género	igualdad poco selectiva
Q3	Materiales más populares	agregación total
Q4	Historial de un usuario	igualdad + orden
Q5	Materiales en un rango de años	rango (BETWEEN)

Metodología: EXPLAIN (ANALYZE, BUFFERS), 6 corridas (se descarta la fría, mediana de 5). DISCARD ALL entre corridas. Escenario "sin índices" con 05_drop_indexes.sql. Se mide con y sin índices en las 4 bases.

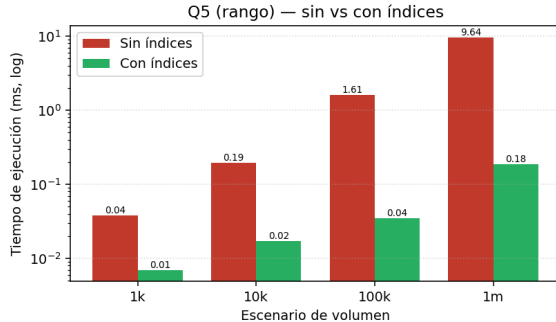
Resultado global (base 1M)



	Sin	Con
Q1 (autor)	7.19	0.08
Q5 (rango)	9.64	0.19
Q4 (historial)	131.05	8.00
Q2 (género)	34.01	22.41
Q3 (populares)	2335.55	2345.02

3 ganan mucho, 1 poco, 1 nada.

Q5 — La consulta por rango (caso estrella)



WHERE Anio_Publicacion BETWEEN 1900 AND 1905

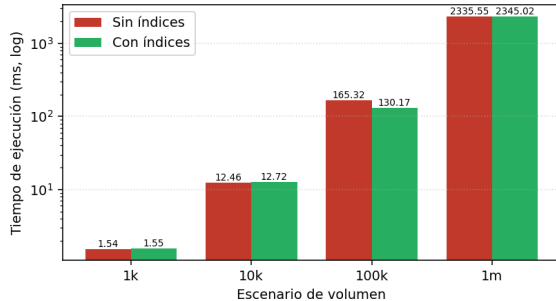
Sin índice: Seq Scan de toda la tabla. Crece con el tamaño.

Con índice: Bitmap Index Scan, toca solo el tramo del rango.

⇒ **52x más rápido** en 1M, y casi plano al escalar.

Q3 — Cuando el índice NO ayuda

Q3 (populares) — sin vs con índices



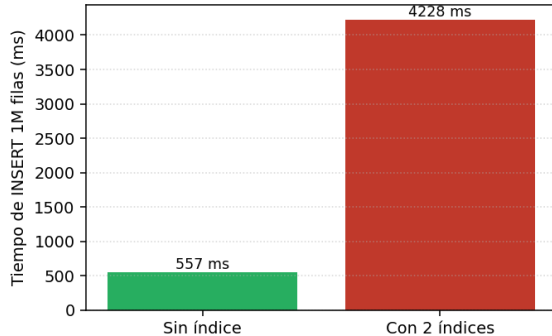
Agregación **total** de likes, lecturas y reseñas: no hay filtro selectivo.

El motor debe leer las tablas completas con o sin índice.

2335 ms vs 2345 ms → 1.0×. Un índice solo ayuda si permite *evitar* leer filas.

El índice no es gratis

Costo de escritura: el índice no es gratis



Escrituras: insertar 1M filas con 2 índices que mantener = **7.6x más lento** (557 ms → 4228 ms).

Lectura no selectiva forzada: usar el índice cuando se lee ~100 % de la tabla = **2.4x más lento** que el Seq Scan (16.6 ms → 40.5 ms).

Por eso el planner lo evita: es la decisión correcta.

- Implementación completa del modelo del Hito 1: tablas, restricciones, triggers y vistas.
- Cuatro volúmenes reproducibles, hasta $\sim 3.86\text{M}$ de filas.
- El experimento confirma los tres comportamientos del índice: gran beneficio (hasta $91\times$), moderado ($1.5\times$) y nulo ($1.0\times$); más su costo en escritura ($7.6\times$).
- **Indexar es una decisión por consulta y por selectividad**, no una receta universal.
- Por encima del millón: la arquitectura cliente-servidor basta para consultas selectivas bien indexadas; las agregaciones totales son el límite.

¿Preguntas?

<https://github.com/SefedeamU/Proyecto-BD-Hito-2>